

SECTION 1: C# BASICS

Q1. What is the difference between var, dynamic, and object in C#?

Answer: var is compile-time type inference — the type is locked at compile time and is fully type-safe. dynamic bypasses the compiler entirely; type resolution happens at runtime via the DLR, meaning no IntelliSense and runtime exceptions if members don't exist. object is the base type of all types in .NET; it requires boxing for value types and casting to retrieve the original type. Use var for clean local declarations, object rarely in modern code, and dynamic only for COM interop or reflection-heavy scenarios — never as a lazy alternative to proper typing.

```
var x = "hello";           // string at compile time, stays string
dynamic y = "hello";      // resolves at runtime - y.NonExistent() compiles but
                           // crashes
object z = "hello";       // needs (string)z to use as string
```

Q2. What is the difference between const, readonly, and static readonly?

Answer: const is a compile-time constant — the value is inlined at every usage site. It must be assigned at declaration and can only be primitive types or strings. readonly is a runtime constant — set in the declaration or the constructor and then immutable. Can be any type. static readonly is shared across all instances and set once. The critical danger of const: if a library changes a const value, all consumers must recompile to see the new value because the old value was baked into their assembly. static readonly avoids this because the value is read at runtime.

```
const double Pi = 3.14159;           // compile-time, inlined everywhere
readonly DateTime StartTime = DateTime.Now; // runtime, set in constructor
static readonly int MaxRetries = 5;   // shared, set once
```

Q3. What is the difference between value types and reference types?

Answer: Value types (int, double, bool, struct, enum) store their data directly. When assigned to a variable or passed to a method, a copy is made. Reference types (class, string, array, interface) store a reference (pointer) to data on the heap. Assignment copies the reference — both variables point to the same object. The heap/stack distinction is nuanced: value types live on the stack when they are local variables, but live inline on the heap when they are fields of a class instance.

```
int a = 5; int b = a; b = 10; // a is still 5 - copied
var l1 = new List<int>{1,2}; var l2 = l1; l2.Add(3); // l1 also has 3 - same
object
```

Q4. Why is string a reference type but behaves like a value type?

Answer: string is defined as 'public sealed class String' — a reference type whose object lives on the heap. However, it is immutable: every operation that appears to modify a string (concatenation, replace) actually returns a new string object, leaving the original untouched. The == operator is overloaded to compare characters, not references. These two facts make string feel like a value type in everyday code. The reference-type nature shows in three places: null is allowed, allocation pressure in loops can hurt performance (use StringBuilder), and ReferenceEquals can distinguish two strings with the same content.

```
string s1 = "hello"; string s2 = "hello";
Console.WriteLine(s1 == s2);           // True - value comparison
```

```

Console.WriteLine(ReferenceEquals(s1, s2));    // True - string interning
string s3 = new string("hello".ToCharArray());
Console.WriteLine(ReferenceEquals(s1, s3));    // False - different objects

```

Q5. What is boxing and unboxing and why does it matter?

Answer: Boxing wraps a value type in an object on the heap. Unboxing extracts it back. Both involve a heap allocation and a copy. The cost is GC pressure in hot paths. Generics were introduced specifically to eliminate boxing in collections — `List<int>` stores ints directly; the old `ArrayList` boxed every integer. Avoid boxing in loops or high-throughput code.

```

int x = 42;
object o = x;           // boxing - heap allocation
int y = (int)o;         // unboxing - cast and copy

```

Q6. What is the difference between `is`, `as`, and a direct cast?

Answer: `'is'` performs a pattern match check — returns `bool`, no exception, modern usage includes pattern matching. `'as'` attempts a cast and returns `null` on failure — only for reference and nullable types, never throws. A direct cast `(Type)x` throws `InvalidCastException` on failure. In modern C# 8+, prefer `'is'` with pattern matching: `if (obj is Customer c) { use c }`. Avoid `'as'` followed by a null check — `'is'` with pattern binding does both in one step.

```

if (obj is Customer c) { c.Name = "Shahid"; }    // best - pattern match
var c = obj as Customer; // null if not Customer - no exception
var c = (Customer)obj;   // throws if not Customer

```

Q7. What are nullable reference types in C# 8+ and why were they introduced?

Answer: Nullable reference types allow the compiler to track whether a reference type might be null. With the feature enabled, `'string'` means not-null and `'string?'` means might-be-null. The compiler warns when you dereference a nullable without a null check. They were introduced to eliminate the 'billion dollar mistake' — `NullReferenceExceptions` that are only caught at runtime. For a greenfield project, always enable: `<Nullable>enable</Nullable>` in the project file. This changes reference semantics from 'anything might be null' to 'null is explicitly declared'.

```

string name = null;    // warning - non-nullable assigned null
string? name = null;   // OK - explicitly nullable
Console.WriteLine(name.Length); // warning - possible null dereference

```

Q8. Explain the difference between `string` and `StringBuilder`. When do you use each?

Answer: `string` is immutable — every concatenation creates a new string object and discards the old one. For a loop of `N` concatenations, you create `N` strings and $O(N^2)$ total work. `StringBuilder` uses a mutable internal buffer — `Append` modifies in place and `ToString` at the end produces one string. Rule: use string interpolation or `+` for 2-3 concatenations; use `StringBuilder` for 5+ concatenations in a loop or when building strings dynamically. Modern C# string interpolation is compiled efficiently for small cases.

```

// Bad in a loop
string result = "";
for (int i = 0; i < 10000; i++) result += i;    // 10,000 allocations

```

```
// Good
var sb = new StringBuilder();
for (int i = 0; i < 10000; i++) sb.Append(i);
string result = sb.ToString();    // one allocation
```

Q9. What is the difference between ref, out, and in parameters?

Answer: ref: passes the variable by reference — caller's variable is modified. Must be initialized before the call. out: passes by reference but the method MUST assign the value inside. The caller does not need to initialize first. in: passes by reference but read-only — the method cannot modify it. Used to avoid copying large structs while guaranteeing immutability. Default (no modifier): value types are copied, reference types pass the reference by value.

```
void Inc(ref int x) { x++; }

bool TryParse(string s, out int r) { r = int.Parse(s); return true; }

double Calc(in LargeStruct d) { return d.Value; } // read-only, no copy
```

Q10. What is string interning in .NET?

Answer: The CLR maintains an intern pool — a single shared instance of each unique string literal compiled into an assembly. Identical string literals in code share the same object in memory. This is why ReferenceEquals returns true for two 'hello' literals even though they are reference types. Runtime strings created via concatenation or new string() are NOT interned by default. You can force interning with string.Intern(). This exists for memory savings — if your code has 10,000 occurrences of the string 'INFO', they share one object. Do not write code that depends on interning behaviour.

```
string s1 = "hello";
string s2 = "hello";

Console.WriteLine(ReferenceEquals(s1, s2));    // True - interned

string s3 = new string("hello".ToCharArray());

Console.WriteLine(ReferenceEquals(s1, s3));    // False - not interned
```

Q11. What is the purpose of the sealed keyword in C#?

Answer: sealed on a class prevents inheritance — no other class can extend it. sealed override on a method prevents further overriding in subclasses. Two reasons to use it: design intent (this class is not designed for extension) and performance (the JIT can devirtualize method calls on sealed types, skipping the virtual dispatch table lookup). In .NET 8, many internal BCL types are sealed for exactly this performance reason. You should seal classes when you have no intention of them being subclassed.

```
sealed class Money { }                                // cannot inherit

class Base { public virtual void M() { } }

class Child : Base { public sealed override void M() { } } // no further override
```

Q12. What is the difference between throw and throw ex in a catch block?

Answer: throw; preserves the original stack trace — the stack points to where the exception originally occurred. throw ex; resets the stack trace to the current location — you lose the original source of the error, making debugging much harder. Always use throw; in catch blocks. The only time you write 'throw someException;' is when someException is a NEW exception you constructed yourself, typically wrapping the original as an inner exception.

```
catch (Exception ex) {
```

```

        throw;           // preserves stack trace — always do this
        throw ex;        // resets stack trace — never do this for rethrow
    }

    // Wrapping is fine:
    catch (SqlException ex) {
        throw new DataAccessException("Failed", ex); // new exception, original as
        inner
    }

```

SECTION 2: OBJECT-ORIENTED PROGRAMMING

Q13. What are the four pillars of OOP?

Answer: Encapsulation: hiding internal state behind public methods so invariants are always enforced. Abstraction: hiding complexity behind contracts — interfaces or abstract classes — so callers depend on what something does, not how. Inheritance: a class reuses and extends another class — IS-A relationship, models shared behaviour. Polymorphism: same method call dispatches to different implementations at runtime based on the actual object type — enabled by virtual/override.

Q14. What is the difference between an abstract class and an interface?

Answer: Abstract class: can have fields, state, constructors, and full method implementations. A class can inherit from only one abstract class. Use when you want to share implementation and state. Interface: defines a contract — methods and properties. No state. A class can implement many interfaces. Use when you want to define a capability or contract. C# 8+ added default interface methods, blurring the line slightly, but interfaces still should not hold state. Rule: share implementation = abstract class; share contract = interface. Prefer interfaces for testability — easier to mock.

```

abstract class Account {
    public string AccountNumber { get; }           // state
    public abstract decimal Calculate();           // must implement
    public virtual string Summary() => "OK";        // may override
}

interface IWithdrawable {
    void Withdraw(decimal amount); // contract only
}

```

Q15. What is the difference between method overloading and overriding?

Answer: Overloading (compile-time polymorphism): multiple methods with the same name but different parameter signatures in the same class. The compiler picks the right one at compile time based on the argument types. Overriding (runtime polymorphism): a derived class replaces a virtual method from the base class using the same signature. The runtime picks the right one at runtime based on the actual object type. Keywords: no special keyword for overloading; 'virtual' on base and 'override' on derived for overriding.

```
// Overloading
public int Add(int a, int b) => a + b;
public double Add(double a, double b) => a + b;

// Overriding
class Base { public virtual void Speak() => Console.WriteLine("Base"); }
class Dog : Base { public override void Speak() => Console.WriteLine("Woof"); }
```

Q16. What is the difference between override and new on a method in a derived class?

Answer: override participates in virtual dispatch — there is ONE method. Casting to the base type is irrelevant; the runtime always finds the most-derived implementation. new (method hiding) creates TWO separate methods with the same name. The VARIABLE type at compile time determines which method runs — casting to base reveals the base method. This is exactly what the virtual/override/new puzzle tests. Use override for polymorphism; avoid new unless you intentionally want two separate methods.

```
class A { public virtual void M() => Console.WriteLine("A"); }
class B : A { public override void M() => Console.WriteLine("B"); }
class C : A { public new void M() => Console.WriteLine("C"); }
```

```
A b = new B(); b.M();    // "B" — override, virtual dispatch
A c = new C(); c.M();    // "A" — new, variable type decides
```

Q17. What is the exact output of this code and why? class A { public virtual double P { get; set; } } class Ao2 : A { public override double P { get; set; } } class An2 : A { public new double P { get; set; } } class A2 : A { public double P { get; set; } }

Answer: var a2 = new A2(){ P = 2 }; Console.WriteLine(a2.P); prints 2. ((A)a2).P prints 0. var ao2 = new Ao2(){ P = 2.1 }; ao2.P prints 2.1; ((A)ao2).P prints 2.1. var an2 = new An2(){ P = 2.2 }; an2.P prints 2.2; ((A)an2).P prints 0. Reason: Ao2 uses override — ONE property. Cast is irrelevant, virtual dispatch always finds Ao2's P. An2 and A2 use new/no-modifier — TWO separate properties with separate backing fields. The object initializer sets the visible property (An2's P or A2's P). A's P is never assigned so returns the default for double which is 0.

```
// A2: compiler warning CS0108 — behaves like 'new'
// Both An2 and A2: two separate P properties in memory
// override: one P, cast irrelevant
// new/no-modifier: two Ps, cast picks which one
```

Q18. What is the Liskov Substitution Principle?

Answer: A subclass must be usable wherever its parent is expected without breaking caller expectations. If your code operates on a BankAccount and calling Withdraw() works, then passing a FixedDepositAccount (which throws NotSupportedException on Withdraw) breaks the principle. The hierarchy is wrong — do not make FixedDepositAccount extend BankAccount. Instead separate the capability: create IWithdrawable and only classes that can withdraw implement it. Violations show up as NotSupportedException, empty method bodies, or unexpected exceptions in methods that accepted the base type.

Q19. What is composition over inheritance and when do you apply it?

Answer: Instead of inheriting behaviour from a parent class, inject the behaviour as a dependency (a collaborator). Inheritance is a tight compile-time single-parent relationship. Composition is flexible — swap implementations at runtime, test with mocks, combine multiple capabilities without a combinatorial explosion of subclasses. Apply it whenever you find yourself creating classes like `FraudCheckedSavingsAccount` that combine concerns. Instead: inject `IFraudChecker` and `IInterestCalculator` into a `BankAccount`. Use inheritance only for true IS-A relationships where you genuinely override behaviour.

Q20. Explain all five SOLID principles with a one-line example for each.

Answer: S — Single Responsibility: a class has one reason to change. `OrderProcessor` should not also send emails. O — Open/Closed: extend with new classes, not by editing existing ones. Add a new payment type by creating a new `IFeeStrategy`, not by adding another if-branch. L — Liskov Substitution: subclasses honor the parent's contract. `FixedDepositAccount` should not inherit `BankAccount` if it cannot withdraw. I — Interface Segregation: many small interfaces beat one fat one. Do not force `FixedDeposit` to implement `Withdraw`. D — Dependency Inversion: depend on abstractions. Inject `IOrderRepository`, not `SqlOrderRepository`.

Q21. What are access modifiers in C# and what does each mean?

Answer: `public`: accessible from anywhere. `private`: same class only — the default for members. `protected`: same class and derived classes. `internal`: same assembly (project) only — the default for top-level types. `protected internal`: same assembly OR any derived class (union). `private protected`: same assembly AND derived classes (intersection), introduced in C# 7.2. Most members should start `private` and only be widened when needed.

Q22. What is a record in C# and how does it differ from a class?

Answer: A record (C# 9+) is a reference type designed for immutable data with value-based equality. The compiler auto-generates: `Equals`, `GetHashCode`, `==`, `!=`, `ToString`, `Deconstruct`, and with-expression support based on all declared properties. A class has reference equality by default — two different objects with the same data are not equal. A record with the same data IS equal. Records can inherit from other records only. Use records for DTOs, events, value objects, API responses — anything you compare by value. Use classes for services and stateful entities.

```
record Customer(string Name, string Email);
var c1 = new Customer("Shahid", "a@b.com");
var c2 = new Customer("Shahid", "a@b.com");
Console.WriteLine(c1 == c2); // True - value equality
var c3 = c1 with { Email = "new@b.com" }; // non-destructive mutation
```

Q23. Can a record inherit from another record? What are the rules?

Answer: Yes. A record can inherit from another record only — not from a plain class, and a class cannot inherit from a record. The auto-generated equality includes an `EqualityContract` property (the runtime Type). Two records are only equal if they have the same `EqualityContract` (same actual type) AND the same property values. So a `Person` record and an `Employee` record that extends it are never equal even if `Employee` has the same `Name`. `record struct` (C# 10+) cannot inherit because structs do not support inheritance.

```
record Person(string Name);
record Employee(string Name, string Dept) : Person(Name);
```

```

Person p = new Employee("Shahid", "Arch");
Person p2 = new Person("Shahid");
Console.WriteLine(p == p2);    // False - different EqualityContract

```

Q24. What is the diamond problem in OOP and how does C# handle it?

Answer: The diamond problem: class A defines method M. Classes B and C both inherit A and override M. Class D inherits both B and C. Which M does D get? C# avoids this entirely by prohibiting multiple class inheritance — a class can have only one base class. You can implement multiple interfaces. In C# 8+, default interface methods can reintroduce an ambiguity: if two interfaces provide a default implementation of the same method and a class implements both, the compiler forces you to disambiguate explicitly.

```

interface IA { void M() => Console.WriteLine("A"); }
interface IB { void M() => Console.WriteLine("B"); }
class C : IA, IB {
    public void M() => ((IA)this).M();    // must disambiguate
}

```

Q25. What is the difference between a struct and a class for performance-sensitive code?

Answer: Structs are value types: allocated inline (stack for locals, inline in containing type for fields). No heap allocation, no GC pressure. Ideal for small, immutable, logically value-like data (Money, Point, CurrencyPair). Classes are reference types: heap allocation, GC managed, identity semantics. Structs should be small (under 16 bytes ideally), immutable (use readonly struct), and free of reference type fields. Large structs hurt performance by copying on every assignment and method call — use 'in' parameters to pass them by reference without copying. Default to class; reach for struct when profiling shows GC pressure.

Q26. What are static members and what are their pitfalls?

Answer: Static members belong to the type, not any instance. Static fields are shared across all instances and persist for the application's lifetime. Static methods can only access other static members. Pitfalls: static mutable state is effectively global state — hard to test (you cannot inject a mock), creates hidden coupling, and causes thread-safety issues in concurrent applications. Use static for pure utility functions and for truly shared immutable data (Math.PI). For everything else, prefer instance members injected via DI.

Q27. What is constructor chaining and how do you use it in C#?

Answer: Constructor chaining lets one constructor delegate to another, reducing duplication. Use 'this' to call another constructor in the same class, or 'base' to call the parent class constructor. The chained constructor always runs first before the body of the calling constructor.

```

class Account {
    public Account(string id) : this(id, 0) { }    // calls the two-param ctor
    public Account(string id, decimal balance) { /* main setup */ }
}

class SavingsAccount : Account {
    public SavingsAccount(string id) : base(id, 100) { }    // calls parent
}

```

SECTION 3: ASYNC, TASKS, AND CONCURRENCY

Q28. Why would you use async/await over working with Task directly?

Answer: async/await does not replace Task — it is syntax for working with tasks ergonomically. The real comparison is async/await versus manual task handling using `.ContinueWith`, `.Result`, or `.Wait`. Reasons to prefer async/await: code reads top-to-bottom like synchronous code (no callback nesting); exception handling works naturally with try/catch (no `AggregateException` unwrapping); the compiler builds the state machine correctly handling locals, continuations, and exceptions; `SynchronizationContext` is captured and resumed properly; and it is non-blocking, releasing the thread during I/O. Use Task directly (without async) only for returning cached values via `Task.FromResult`, bridging callbacks with `TaskCompletionSource`, or composing tasks with `Task.WhenAll` where no awaiting is needed.

```
// Without await - unreadable
return _http.GetAsync(url).ContinueWith(t => t.Result.Content.ReadAsStringAsync()
    .ContinueWith(t2 => Deserialize(t2.Result))).Unwrap();

// With await - clear
var response = await _http.GetAsync(url);
var json = await response.Content.ReadAsStringAsync();
return Deserialize(json);
```

Q29. What does async/await actually do under the hood?

Answer: The C# compiler transforms an async method into a state machine — a struct implementing `IAsyncStateMachine`. Each await is a state transition. When the method hits an await on an incomplete Task, the state machine captures all local variables, registers a continuation, and returns control to the caller. No thread is blocked. When the awaited task completes, a thread pool thread picks up the continuation and resumes from the captured state. This is why async/await is about asynchrony, not threads — it releases threads during I/O waits rather than creating new ones.

Q30. What type is 'res' in this code and could it deadlock? public async Task<object> T1(){ ... } public async Task T2(){ var res = T1().Result; }

Answer: `T1()` returns `Task<object>`. Calling `.Result` blocks the current thread and unwraps the completed task, giving an object. So `res` is of type `object`. Deadlock risk: yes, in environments with a single-threaded `SynchronizationContext` such as classic ASP.NET on .NET Framework or WPF. The mechanism: `.Result` blocks the calling thread; the continuation inside `T1` needs to resume on the same captured context; the context is busy blocking; the continuation never runs; the task never completes. In ASP.NET Core there is no `SynchronizationContext` so this specific deadlock does not occur, but blocking on async still wastes a thread pool thread and can cause thread starvation. Rule: go async all the way up. Never block on async code.

```
// Wrong
var res = T1().Result;    // blocks

// Right
```



```
var res = await T1();           // non-blocking
```

Q31. What is ConfigureAwait(false) and when should you use it?

Answer: By default, await captures the current SynchronizationContext and resumes the continuation on it. ConfigureAwait(false) tells the runtime: do not capture the context, resume on any available thread pool thread. Three reasons to use it in library code: prevents deadlocks in hosts that have a single-threaded SynchronizationContext; avoids the overhead of context-switching back to the original context; and libraries should not impose threading requirements on callers. In ASP.NET Core there is no SynchronizationContext so it is mostly a stylistic discipline, but it is correct hygiene for any code that may be consumed as a library.

```
// Library code - always ConfigureAwait(false)
var data = await _http.GetStringAsync(url).ConfigureAwait(false);
```

Q32. What is the difference between Task.WhenAll and Task.WhenAny?

Answer: Task.WhenAll returns a task that completes when ALL provided tasks complete. If any task faults, WhenAll faults with an AggregateException containing all failures. Use when you need all results before proceeding. Task.WhenAny returns a task that completes when the FIRST provided task completes (success or failure). Use for racing a primary against a fallback or timeout. Always check the result for exceptions when using WhenAny.

```
// Run both concurrently, wait for all
var (balance, history) = await Task.WhenAll(GetBalanceAsync(), GetHistoryAsync());

// Race primary and timeout
var first = await Task.WhenAny(primary, Task.Delay(5000));
if (first == primary) { /* success */ }
```

Q33. What is the difference between async/await and multithreading?

Answer: async/await is about asynchrony, not parallelism. It does not create threads. It releases the current thread during I/O waits (network, disk) so the thread can do other work. The continuation may resume on the same or a different thread pool thread. Multithreading (Task.Run, Parallel.ForEach, Thread) uses multiple threads simultaneously for CPU-bound parallel work. Confusing the two is a common mistake: wrapping CPU work in async does not make it parallel; you need Task.Run. Awaiting I/O does not create threads; it just avoids blocking one.

Q34. What is CancellationToken and how do you propagate it correctly?

Answer: CancellationToken is a cooperative cancellation mechanism. A CancellationTokenSource creates a token. You pass the token to long-running operations, which check it at regular intervals and stop gracefully. The token is passed down through every async call in the chain — not stopping at the first level. Check with ThrowIfCancellationRequested() or pass to I/O operations that accept it. Catch OperationCanceledException at the top level. ASP.NET Core provides HttpContext.RequestAborted which fires when the client disconnects.

```
public async Task ProcessAsync(CancellationToken ct) {
    foreach (var item in items) {
        ct.ThrowIfCancellationRequested();
        await _db.SaveAsync(item, ct);
    }
}
```

```
}
```

Q35. What is the difference between Task and ValueTask?

Answer: Task is a heap-allocated reference type. Every async method that returns Task allocates at least one object. ValueTask is a struct — when the method completes synchronously (common with cache hits), there is zero allocation. Use ValueTask in hot paths where the async method often has a result immediately available. Caution: do not await a ValueTask more than once and do not store it for later — it is designed for single-consumption. Default to Task. Add ValueTask only after profiling shows allocation pressure.

```
public ValueTask<string> GetAsync(int id) {  
    if (_cache.TryGetValue(id, out var v)) return new ValueTask<string>(v);  
    return new ValueTask<string>(LoadAsync(id)); // async path  
}
```

Q36. What is IAsyncEnumerable and when would you use it?

Answer: IAsyncEnumerable<T> (C# 8+) is the async equivalent of IEnumerable<T>. It allows async lazy streaming of data. Instead of loading all results into a List and returning it, you yield each item as it becomes available. Use with 'await foreach'. Ideal for streaming large query results from a database row by row without buffering everything into memory, or for processing Kafka/Service Bus messages as a continuous stream. Prevents out-of-memory errors on large datasets and reduces time-to-first-result.

```
async IAsyncEnumerable<Trade> StreamTradesAsync() {  
    await foreach (var row in db.StreamAsync())  
        yield return MapToTrade(row);  
}  
  
await foreach (var trade in StreamTradesAsync()) Process(trade);
```

Q37. What are common causes of thread starvation in async .NET applications?

Answer: Thread starvation occurs when thread pool threads are exhausted — all threads are blocked waiting for tasks, but new work items cannot run because no threads are available. Causes: blocking on async code with .Result or .Wait (each blocked call holds a thread indefinitely); sync-over-async patterns cascading through the call stack; CPU-bound work saturating the thread pool; and tight loops using Task.Delay(0) or spinwaits. Fix: go async all the way up; use await for all I/O; offload CPU work to dedicated threads if needed.

Q38. What is a deadlock and how does it arise in async code with SynchronizationContext?

Answer: A deadlock is two participants waiting for each other indefinitely. In async code: Thread T1 holds a single-threaded SynchronizationContext (UI thread or classic ASP.NET request context). T1 calls .Result on a Task, blocking itself. The Task's continuation needs to run on T1's context to complete. T1 is blocked waiting for completion. The continuation cannot run. Neither side can proceed. The two ingredients: a single-threaded SynchronizationContext, and blocking that thread with .Result or .Wait. Fix: never block on async; use await. Or use ConfigureAwait(false) inside the awaited method so the continuation does not need the captured context.

SECTION 4: LINQ

Q39. What is deferred execution in LINQ and what is the classic trap?

Answer: LINQ queries do not execute when you define them — they build an expression tree or delegate chain. Execution happens when you enumerate the query (foreach, ToList, Count, First). This is deferred execution. Classic trap: you define a query, modify the source collection, then enumerate — the query sees the modified source because it re-reads it on execution. Another trap: defining a query in a using block and enumerating after the resource is disposed — the query fails because the database connection is gone. To take a snapshot, call ToList() immediately.

```
var query = numbers.Where(n => n > 5);    // nothing runs yet
numbers.Add(99);
foreach (var n in query) { }             // includes 99 - query re-evaluated
```

Q40. What is the difference between IEnumerable<T> and IQueryable<T>?

Answer: IEnumerable<T> operates in memory. LINQ-to-Objects. The entire sequence is pulled into the app; filtering and mapping happen in C# code. IQueryable<T> builds an expression tree that a query provider (EF Core) translates to SQL. Filtering runs at the database. Critical performance difference: calling Where on an IQueryable adds a SQL WHERE clause; calling Where on an IEnumerable after calling AsEnumerable() loads ALL rows into memory and filters in C#. Always filter on IQueryable before materializing.

```
// Good - SQL WHERE Amount > 1000
var q = dbContext.Trades.Where(t => t.Amount > 1000).ToList();

// Bad - loads ALL trades into memory, then filters
var q = dbContext.Trades.AsEnumerable().Where(t => t.Amount > 1000).ToList();
```

Q41. What is the difference between Select and SelectMany?

Answer: Select projects each element to one result — 1-to-1 mapping. SelectMany projects each element to a sequence and then flattens all sequences into one — 1-to-many then flatten. Think of SelectMany as Select + flatten. Equivalent to a nested foreach or a SQL CROSS APPLY / INNER JOIN with a collection.

```
// Select - nested sequences
customers.Select(c => c.Orders)           // [[o1,o2],[o3]]

// SelectMany - flat sequence
customers.SelectMany(c => c.Orders)       // [o1, o2, o3]
```

Q42. What is the difference between First, FirstOrDefault, Single, and SingleOrDefault?

Answer: First: returns the first match; throws if empty. FirstOrDefault: returns the first match or default(T) (null for reference types) if empty. Single: returns the only match; throws if empty OR if more than one match. SingleOrDefault: returns the only match or default(T) if empty; throws if more than one match. Use Single when exactly one result is contractually expected (primary key lookup) — extra matches indicate data corruption. Use First when you just want any match.

Q43. Explain GroupBy with a banking example.

Answer: GroupBy partitions a sequence by a key. Each group has a Key and a sequence of items in that group. Equivalent to SQL GROUP BY. With EF Core it translates to a SQL GROUP BY clause.

```
var byCurrency = trades.GroupBy(t => t.Currency);
foreach (var g in byCurrency)
    Console.WriteLine($"{g.Key}: total = {g.Sum(t => t.Amount)}");
// USD: total = 300
// EUR: total = 50
```

Q44. What is the difference between List<T>.Sort() and LINQ OrderBy()?

Answer: List.Sort() mutates the original list in place and returns void. It uses IntroSort which is unstable — equal elements may reorder. LINQ OrderBy returns a new IEnumerable leaving the original unchanged. It is stable — equal elements preserve their original relative order. OrderBy also has deferred execution. Stability matters in financial systems: if you sort trades by currency but want them to stay in chronological order within each currency, you need a stable sort. Use ThenBy for secondary keys. Default direction is ascending for both; use OrderByDescending for descending.

Q45. What does Aggregate do in LINQ?

Answer: Aggregate applies a function cumulatively to a sequence, optionally starting from a seed value. It is the general-purpose fold/reduce operation. Sum, Max, Min are all special cases of Aggregate. Use it when no built-in aggregation method exists for your computation.

```
// Running product
int product = new[]{1,2,3,4}.Aggregate(1, (acc, x) => acc * x); // 24

// Build a string
string csv = names.Aggregate((a, b) => a + "," + b);
```

Q46. What is the difference between Any and All in LINQ?

Answer: Any() with no argument returns true if the sequence has any elements. Any(predicate) returns true if at least one element satisfies the predicate — short-circuits on first match. All(predicate) returns true only if every element satisfies the predicate — short-circuits on first failure. Any and All on IQueryable translate to SQL EXISTS and NOT EXISTS respectively. Both are more efficient than Count() > 0 for existence checks.

```
bool hasHighValue = trades.Any(t => t.Amount > 100000);
bool allSettled = trades.All(t => t.Status == TradeStatus.Settled);
```

Q47. What is the yield keyword and how does it work?

Answer: yield return inside a method makes it an iterator — the compiler generates a state machine implementing IEnumerable<T>. Execution is lazy: the method body does not run until enumeration starts. At each yield return, the value is returned to the caller and execution pauses. It resumes from the same point on the next MoveNext() call. yield break terminates early. Key pitfall: resources stay open for the entire enumeration — if you yield from a database reader, the connection stays open. Also, yield return cannot be used inside a try block that has a catch (only try/finally is allowed).

```
IEnumerable<int> Evens(int max) {
    for (int i = 0; i <= max; i += 2)
```

```
        yield return i;    // pauses here, resumes on next call
    }
}
```

Q48. What is the difference between Skip/Take and keyset pagination? Which is better for large datasets?

Answer: Skip/Take (offset-based pagination): SKIP N rows TAKE M rows. Simple to implement. Problem: the database must scan and discard the first N rows every time, getting slower as the page number increases. Also susceptible to 'page drift' — if rows are inserted between page requests, items shift and you see duplicates or skip items. Keyset pagination (cursor-based): instead of Skip, use WHERE Id > lastSeenId ORDER BY Id TAKE M. The database uses the index to jump directly to the right row. O(log n) regardless of page number. Consistent results. Downside: you cannot jump to an arbitrary page number. For financial systems with large datasets (trade history), keyset pagination is always the correct choice.

```
// Offset — gets slower with each page
Skip(page * size).Take(size)

// Keyset — always fast
.Where(t => t.Id > lastSeenId).OrderBy(t => t.Id).Take(size)
```

SECTION 5: MIDDLEWARE AND FILTERS

Q49. What is middleware in ASP.NET Core?

Answer: Middleware is a chain of components that process every HTTP request in a pipeline. Each component receives the request, optionally modifies it, calls the next component via 'await next()', and then optionally processes the response on the way back. The pipeline is nested — each middleware wraps the next like nested boxes. Middleware registered first runs outermost: first on the way in, last on the way out. Registered with app.Use, app.Run (terminal), or app.Map (branching by path).

Q50. Why would you write custom middleware?

Answer: For cross-cutting concerns that apply to all or most requests and do not belong in any single controller: request/response logging with timing and correlation IDs for distributed tracing; tenant resolution from subdomain or header in multi-tenant applications; global exception handling that returns consistent RFC 7807 ProblemDetails; rate limiting and throttling; custom authentication for non-standard schemes such as API keys or HMAC-signed requests; and audit logging for compliance. Middleware does this once. Putting it in controllers means repeating it hundreds of times.

Q51. Why does the order of middleware registration matter? Give an example.

Answer: Middleware runs in registration order. Wrong order causes subtle bugs. Most critical: UseAuthentication must come before UseAuthorization. Authentication populates HttpContext.User; Authorization checks it. If authorization runs first, HttpContext.User is empty and every authenticated endpoint returns 401 even with a valid token. Standard correct order: UseExceptionHandler first (outermost, catches everything), then HTTPS redirection, static files, routing, CORS, authentication, authorization, map endpoints.

```

app.UseExceptionHandler(); // first - catches everything below
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseCors();
app.UseAuthentication(); // must be before Authorization
app.UseAuthorization();
app.MapControllers();

```

Q52. If middleware 2 in a 3-middleware pipeline throws an exception, where does it go?

Answer: Back up to middleware 1, not forward to middleware 3. The pipeline is nested function calls, not a flat queue. Each middleware calls the next via 'await next()'. If M2 throws before calling next(), M3 never runs. The exception unwinds back through M1's await next() call exactly like any other thrown exception in a method call chain. M3 is unreachable. This is why the global exception handler is registered first — it sits on the outside of everything and can catch exceptions from any middleware below it.

```

app.Use(async (ctx, next) => {
    try { await next(); } // M2 throws here
    catch { /* M1 catches */ }
});
app.Use(async (ctx, next) => { throw new Exception(); }); // M2
app.Use(async (ctx, next) => { await next(); }); // M3 - never reached

```

Q53. What is the difference between middleware and MVC filters?

Answer: Middleware runs for ALL HTTP requests — static files, gRPC, SignalR, health checks, and MVC. It sees raw HttpContext but does not know which controller or action will handle the request. Filters run INSIDE MVC, after routing has matched a controller action. They have access to MVC-specific context: the action descriptor, action arguments, model state, and the result object. Rule: if the concern applies to all requests including non-MVC ones, use middleware. If you need to know which controller or action is running or need access to model binding results, use a filter.

Q54. What are the types of MVC filters and when do you use each?

Answer: Authorization filter: first to run, before model binding. Short-circuit with 401/403 for unauthorized access. Resource filter: runs after authorization, before model binding. Used for response caching — skip the rest of the pipeline for cached responses. Action filter: runs before and after the action method, after model binding. Use for logging which action ran, modifying model state, measuring action duration. Exception filter: catches unhandled exceptions thrown by actions or other filters. Use for translating domain exceptions to HTTP responses for specific controllers. Result filter: runs before and after the result is executed. Use for adding headers to all responses from a controller.

Q55. Why do exception filters exist when there is already global exception handling middleware?

Answer: Global exception middleware catches everything but handles it generically. Exception filters are scoped to specific controllers and have MVC context (ObjectResult, IActionResult, model state). They translate domain exceptions into specific, structured HTTP responses without cluttering every

action with try/catch. For example: an exception filter on PaymentsController translates InsufficientFundsException to a 422 with a structured body containing the requested and available amounts. This keeps controller actions clean and puts exception translation logic in one place per controller. Middleware cannot do this because it lacks MVC context.

```
public class DomainExceptionFilter : IExceptionHandler {
    public void OnException(ExceptionContext ctx) {
        if (ctx.Exception is InsufficientFundsException ex) {
            ctx.Result = new ObjectResult(new { error = ex.Message })
                { StatusCode = 422 };
            ctx.ExceptionHandled = true;
        }
    }
}
```

Q56. Where would you place a global exception handler — middleware or a filter? Why?

Answer: Middleware, registered first. Exception filters only catch exceptions thrown during MVC execution — action methods and other filters. They do not catch exceptions thrown in non-MVC middleware or in middleware that runs before MVC. Global exception middleware sits at the outermost layer and catches everything: exceptions from any middleware, routing failures, and MVC exceptions. A filter can then handle domain-specific translation within a controller hierarchy. Both play complementary roles: middleware catches everything; filters handle domain-specific translation.

SECTION 6: GARBAGE COLLECTION AND MEMORY

Q57. How does .NET's garbage collector work and what are generations?

Answer: The GC uses generational collection based on the observation that most objects die young. Gen 0: newly allocated objects, collected frequently, very fast. Gen 1: objects that survived one Gen 0 collection, acts as a buffer. Gen 2: long-lived objects collected rarely. When Gen 0 fills, a Gen 0 collection runs. If that is insufficient, Gen 1 is collected. Full collection includes Gen 2 (slowest). The GC marks all reachable objects from roots (stack, statics, GC handles), sweeps unreachable memory, and compacts surviving objects together to eliminate fragmentation.

Q58. What is the Large Object Heap and why does it matter?

Answer: Objects 85,000 bytes or larger go on the Large Object Heap (LOH), a separate heap from Gen 0/1/2. The GC historically did not compact the LOH (compacting large objects is expensive). This causes fragmentation — gaps that smaller objects cannot fill, leading to apparent memory leaks and out-of-memory exceptions despite reported available memory. LOH is collected only during Gen 2 collections. For applications that frequently allocate large arrays (parsing, serialization), pool them with `ArrayPool<T>` rather than allocating new ones each time.

```
// Bad — allocates a new large array every call
byte[] buffer = new byte[100_000];

// Good — rents from pool, returns after use
byte[] buffer = ArrayPool<byte>.Shared.Rent(100_000);
```

```
try { /* use */ } finally { ArrayPool<byte>.Shared.Return(buffer); }
```

Q59. What is IDisposable and when must you implement it?

Answer: IDisposable provides deterministic cleanup of unmanaged resources — file handles, database connections, network sockets — that the GC cannot reliably clean up because it does not know about them. Implement IDisposable when your class directly owns unmanaged resources or owns other IDisposable objects. Always wrap IDisposable objects in a using statement or using declaration so Dispose is called deterministically. Failure to dispose exhausts connection pools, leaks file handles, and wastes OS resources.

```
using var conn = new SqlConnection(connStr); // disposed at end of scope
using var reader = await cmd.ExecuteReaderAsync();
```

Q60. What is the difference between a finalizer and Dispose?

Answer: Dispose is called by your code explicitly or via using — deterministic. The finalizer (~ClassName) is called by the GC non-deterministically, potentially milliseconds or hours after the object becomes unreachable. Implement Dispose for cleanup. Implement a finalizer only if you directly hold an unmanaged resource without using SafeHandle — rare in modern code. Always call GC.SuppressFinalize(this) in Dispose so the GC does not run the finalizer unnecessarily. Most user code never needs a finalizer.

```
public void Dispose() {
    if (_disposed) return;
    _stream?.Dispose();
    _disposed = true;
    GC.SuppressFinalize(this);
}
```

Q61. How do memory leaks happen in .NET despite garbage collection?

Answer: Five common causes: 1) Unsubscribed event handlers — the event source holds a reference to the subscriber keeping it alive. 2) Static collections — adding to a static List or Dictionary without ever removing, growing forever. 3) Long-lived caches without eviction — MemoryCache without TTL or size limits. 4) Captured variables in long-lived closures — a closure captures everything in its enclosing scope, extending their lifetime. 5) Unmanaged resources not disposed — the managed wrapper is eventually collected but the underlying OS handle leaks until then. Tools: dotMemory, PerfView, Visual Studio Diagnostic Tools.

Q62. What is Span<T> and ReadOnlySpan<T> and why are they relevant for performance?

Answer: Span<T> is a ref struct that provides a type-safe, bounds-checked view over contiguous memory (arrays, stack memory, native buffers) without allocating new memory. ReadOnlySpan<T> is the immutable variant. They allow parsing and slicing strings and byte arrays without allocating substrings or sub-arrays. Stack-only (cannot be stored on the heap or used in async methods — use Memory<T> for async). Critical for high-throughput scenarios: parsing FIX protocol messages, processing CSV data, reading from network buffers. Eliminates GC pressure in hot paths.

```
ReadOnlySpan<char> input = "USD/JPY/100";
int slash = input.IndexOf('/');
var pair = input[..slash]; // no new string allocated
```


Q63. What is boxing and how do you avoid it?

Answer: Boxing wraps a value type in an object on the heap — allocation + copy. Unboxing extracts it back. Occurs when a value type is assigned to object, an interface variable, or a non-generic collection. Avoid by: using generic collections (`List<int>` not `ArrayList`), avoiding object parameters in hot paths, using constrained generics (where `T : struct`), and using `Span<T>` for buffer manipulation. Modern C# with generics has largely eliminated accidental boxing in well-written code.

```
ArrayList list = new(); list.Add(5); // boxes every int - avoid
List<int> list = new(); list.Add(5); // no boxing - generics
```

SECTION 7: COLLECTIONS, SORTING, AND DATA STRUCTURES

Q64. Why would you choose a Binary Search Tree over a Dictionary?

Answer: Dictionary (hash table) gives $O(1)$ average lookup but has no concept of order. Choose a BST (SortedDictionary, SortedSet in .NET — backed by a red-black tree) when: you need range queries (all keys between X and Y), you need ordered iteration without sorting on every access, you need predictable $O(\log n)$ worst-case performance (Dictionary is $O(n)$ worst case on hash collisions), you need floor or ceiling operations (largest key less than or equal to a given value), or your keys do not hash well. For a single point lookup by ID, Dictionary wins. For ordered or range-based access patterns, BST wins.

Q65. What is the difference between SortedDictionary and SortedList?

Answer: SortedDictionary is backed by a red-black tree. $O(\log n)$ insert, delete, lookup. Better when the collection is frequently modified. SortedList is backed by a sorted array. $O(\log n)$ lookup via binary search, but $O(n)$ insert and delete because it must shift elements. Better when you load data once and read many times — lower memory and better cache locality due to array storage. Both maintain keys in sorted order and support ordered enumeration.

Q66. What is the Big-O complexity of common operations on List, Dictionary, HashSet, and SortedDictionary?

Answer: List: index access $O(1)$, Contains $O(n)$, Add at end $O(1)$ amortized, Insert at index $O(n)$, Sort $O(n \log n)$. Dictionary: lookup $O(1)$ avg $O(n)$ worst, insert $O(1)$ avg, no ordering. HashSet: Contains $O(1)$ avg, Add $O(1)$ avg, no ordering. SortedDictionary (red-black tree): lookup $O(\log n)$, insert $O(\log n)$, delete $O(\log n)$, ordered iteration $O(n)$.

Q67. What is a stable sort and why does it matter in financial systems?

Answer: A stable sort preserves the relative order of elements that compare as equal. If trade A and trade B both have currency USD and you sort by currency, a stable sort keeps A before B if A appeared before B in the original sequence. An unstable sort may swap them. LINQ OrderBy is stable. List.Sort (IntroSort) is unstable. In financial systems: when sorting trades by currency for a report, you typically want them in chronological order within each currency. A stable sort with a currency key gives this without a secondary timestamp key. An unstable sort requires explicit secondary sorting.

Q68. What is the difference between Queue<T>, Stack<T>, and PriorityQueue<T, P>?

Answer: Queue<T>: FIFO (First In First Out). Enqueue adds to the back, Dequeue removes from the front. O(1) for both. Use for task scheduling, BFS, message processing, request buffering. Stack<T>: LIFO (Last In First Out). Push adds to the top, Pop removes from the top. O(1). Use for undo operations, DFS, expression parsing, call stack simulation. PriorityQueue<TElement, TPriority> (.NET 6+): dequeues the element with the lowest priority value first (min-heap). O(log n) enqueue and dequeue. Use for job scheduling by urgency, Dijkstra's algorithm, SLA-based processing.

Q69. What is the difference between HashSet<T> and Dictionary<K, V>?

Answer: HashSet<T> stores unique values only. O(1) average for Contains, Add, Remove. No value associated — you are only tracking membership or uniqueness. Dictionary<K, V> stores key-value pairs. O(1) average for ContainsKey, access, and update. When you need to check whether something exists without needing associated data, HashSet is simpler and more memory-efficient. When you need the value too, use Dictionary.

Q70. What is a ConcurrentDictionary and when do you use it?

Answer: ConcurrentDictionary<K, V> is a thread-safe dictionary optimized for concurrent reads and writes. It uses fine-grained locking (lock striping) rather than a single global lock. Use it when multiple threads read and write to shared dictionary state simultaneously — for example, an in-memory cache in a service registered as singleton. Key methods: TryAdd, TryUpdate, AddOrUpdate, GetOrAdd. GetOrAdd has a subtle issue: the value factory can be called multiple times under contention even if only one result is stored — make the factory idempotent.

Q71. Implement a rate limiter data structure using a sliding window approach.

Answer: Use a Dictionary mapping client ID to a Queue of timestamps. On each request: dequeue timestamps older than the window, check if remaining count exceeds the limit, enqueue the current timestamp if allowed. Thread-safety requires a lock per client entry or a ConcurrentDictionary with per-entry locking. For distributed rate limiting across multiple API instances, store counters in Redis using atomic INCR with TTL.

```
private readonly ConcurrentDictionary<string, Queue<DateTime>> _windows = new();
public bool IsAllowed(string client, int max, TimeSpan window) {
    var q = _windows.GetOrAdd(client, _ => new Queue<DateTime>());
    lock (q) {
        var cutoff = DateTime.UtcNow - window;
        while (q.Count > 0 && q.Peek() < cutoff) q.Dequeue();
        if (q.Count >= max) return false;
        q.Enqueue(DateTime.UtcNow);
        return true;
    }
}
```

Q72. What is the closure variable capture trap in C# and how do you fix it?

Answer: A closure captures a variable by reference, not by value. In a for loop, all closures share the same loop variable. By the time the closures execute, the loop has finished and the variable holds its final value, so all closures produce the same result.

```
// Trap - all print 3
var actions = new List<Action>();
for (int i = 0; i < 3; i++)
    actions.Add(() => Console.WriteLine(i));
actions.ForEach(a => a());    // 3 3 3

// Fix - capture a copy per iteration
for (int i = 0; i < 3; i++) {
    int captured = i;
    actions.Add(() => Console.WriteLine(captured));
}    // 0 1 2
```

SECTION 8: EXCEPTION HANDLING

Q73. Explain try, catch, finally. When does finally NOT run?

Answer: try contains the protected code. catch handles specific exception types — list specific types before general ones (Exception must be last). finally always runs regardless of whether an exception was thrown or caught, even if the catch block rethrows. finally is used for cleanup: closing connections, releasing locks. Cases where finally does NOT run: Environment.FailFast() calls, StackOverflowException in most cases, OS killing the process, and power loss. For 99.9% of application code, finally always runs.

Q74. What are exception filters (the when keyword) and why are they useful?

Answer: The when clause on a catch block adds a condition — the catch only handles the exception if the condition is true. If the condition is false, the exception continues unwinding to the next handler as if the catch did not exist. Critical advantage: when the filter returns false, the stack is NOT unwound — debuggers stop at the original throw site, giving better diagnostic information. This is better than catching and rethrowing, which unwinds the stack first.

```
catch (HttpRequestException ex) when (ex.StatusCode == HttpStatusCode.NotFound)
{ return null; }

catch (HttpRequestException ex) when (ex.StatusCode >=
HttpStatusCode.InternalServerError)
{ await RetryAsync(); }
```

Q75. What makes a good custom exception class?

Answer: A good custom exception: inherits from Exception or a more specific base class; ends with 'Exception' by convention; provides multiple constructors (message-only, message+inner, default); adds domain-specific properties that carry structured information beyond a message string; is thrown for recoverable or domain-meaningful errors, not for control flow; and is documented so callers know to expect it. Custom exceptions allow callers to catch specifically and programmatically access structured error data.

```

public class InsufficientFundsException : Exception {
    public decimal Requested { get; }
    public decimal Available { get; }
    public InsufficientFundsException(decimal req, decimal avail)
        : base($"Insufficient funds: needed {req}, have {avail}") {
        Requested = req; Available = avail;
    }
}

```

Q76. What is AggregateException and when does it arise?

Answer: AggregateException wraps one or more exceptions into a single exception, used by the Task Parallel Library. When a Task faults, the exception is stored in Task.Exception as an AggregateException. Task.WhenAll wraps all faulted tasks' exceptions into one AggregateException. Awaiting a faulted Task unwraps the first inner exception and rethrows it directly — no AggregateException. Blocking with .Result or .Wait throws AggregateException. Use GetAwaiter().GetResult() to block and get the original exception without AggregateException wrapping.

Q77. How do you handle exceptions in async void methods and why are they dangerous?

Answer: async void methods cannot be awaited. Any exception thrown inside them propagates to the SynchronizationContext as an unhandled exception — it is not catchable at the call site and will crash the application or cause undefined behavior. The only legitimate use of async void is event handlers (required by delegate signature). For all other cases, return async Task so the caller can await it and handle exceptions. When you must use async void in event handlers, wrap the body in try/catch.

```

// Dangerous
async void DoWork() { throw new Exception(); } // uncatchable at call site

// Safe
async Task DoWork() { throw new Exception(); } // caller can catch with await

```

Q78. What is structured exception handling and how do you design exception hierarchies?

Answer: Structured exception handling means each layer of the application catches exceptions it can handle and lets others propagate. Design: domain layer throws domain exceptions (InsufficientFundsException, InvalidTradeException). Infrastructure layer translates infrastructure exceptions to domain exceptions (SqlException becomes DataAccessException). Application layer handles domain exceptions, logs them, and converts to API responses. The caller never sees SqlException — that is an implementation detail. This creates clear, testable exception contracts at each boundary.

Q79. What is the difference between fail-fast and graceful degradation in exception handling?

Answer: Fail-fast: detect invalid state immediately and throw early rather than letting bad data propagate and cause confusing downstream errors. Validate at system entry points (constructor arguments, API inputs). Graceful degradation: when a non-critical dependency fails, return a

degraded but functional response rather than propagating failure to the user. For example: if the recommendation service fails, return a product page without recommendations rather than a 500 error. Circuit breakers implement this pattern — after repeated failures, stop calling a dependency and use a fallback.

SECTION 9: AZURE AND CLOUD

Q80. What is the difference between Azure Service Bus, Event Grid, and Event Hubs?

Answer: Service Bus is an enterprise message broker for commands and workflows. Queues give point-to-point guaranteed delivery. Topics give publish/subscribe. Features: sessions for ordered delivery, dead-letter queues, transactions, duplicate detection, scheduled delivery. Use when a message must be processed exactly once and ordering may matter. Event Grid is for reactive event routing — push-based, sub-second latency. Best for system events (blob created, resource changed) and fan-out to many handlers. Best-effort delivery, retries for 24 hours. No ordering guarantees. Event Hubs is for high-throughput event streaming — millions of events per second, Kafka-compatible, events retained and replayable. Use for telemetry, logs, IoT, event sourcing.

Q81. What is partitioning in Azure Service Bus and why does it matter?

Answer: Two concepts. Physical partitioning: spreads a queue or topic across multiple message brokers for higher throughput. Set at creation time, cannot be changed, only available on Standard tier not Premium. Sessions (logical partitioning): group messages by a SessionId so all messages with the same ID are processed in order by a single consumer. Different sessions are processed in parallel by different consumers. For banking: use SessionId = accountId. All trades for account 12345 process in order; trades for account 67890 process concurrently. Sessions give ordering where it matters and parallelism across customers.

Q82. What is a managed identity in Azure and why is it better than a connection string?

Answer: A managed identity is an automatically managed identity in Azure Active Directory assigned to an Azure resource (VM, App Service, Function). The resource uses this identity to authenticate to other Azure services without needing any stored credentials — no connection strings, no client secrets in config files. Two types: system-assigned (tied to the resource, deleted with it) and user-assigned (standalone, shareable). Eliminates the credential rotation problem — the platform manages the identity. Much more secure: no secret to leak, rotate, or accidentally commit to source control.

```
var credential = new DefaultAzureCredential();  
var client = new SecretClient(new Uri(vaultUrl), credential);
```

Q83. How do you secure secrets in a cloud-native .NET application?

Answer: Never store secrets in appsettings.json or plain environment variables. Store secrets in Azure Key Vault. Use managed identities so the application authenticates to Key Vault without credentials. Use the CSI Secrets Store driver to mount secrets as files in Kubernetes pods. Rotate secrets through the platform, not manual deployments. Never log secrets. Audit all access. In development, use user secrets (dotnet user-secrets) which are stored outside the project directory and never committed.

Q84. What is the difference between horizontal and vertical scaling?

Answer: Vertical scaling (scale up): replace the server with a bigger one — more CPU, RAM, faster disk. Simple to implement, requires no application changes. Limited by the largest available machine size. No improvement to availability. Horizontal scaling (scale out): add more instances of the application. Requires stateless design or externalized state (session in Redis, files in blob storage). Improves both throughput and availability. Cloud platforms are designed for horizontal scaling — auto-scaling adds and removes instances based on load metrics.

Q85. What is Azure Kubernetes Service (AKS) and what does it manage for you?

Answer: AKS is a managed Kubernetes service. Azure runs the Kubernetes control plane (API server, etcd, scheduler, controller manager) for free. You pay only for the worker nodes (VMs) in your node pools. AKS handles: control plane patching and upgrades, node OS patching, integration with Azure AD for RBAC, integration with Azure Monitor and Container Insights, integration with Azure Container Registry, automatic certificate rotation, and horizontal pod autoscaling. You still manage: your application deployments, pod definitions, network policies, and node pool sizing.

Q86. How would you achieve zero-downtime deployments in Kubernetes?

Answer: Rolling updates: Kubernetes replaces pods one at a time. New pods must pass readiness probes before old pods are terminated. Ensure readinessProbe is configured correctly. Blue-green deployment: maintain two identical environments. Deploy to green, test, switch traffic via the service selector label, retire blue. Canary deployment: route 5% of traffic to the new version, monitor error rates and latency, progressively increase. Always: database migrations must be backward compatible and run before the new code deploys. Use init containers or a pre-deploy job for migrations.

Q87. What is pod security in Kubernetes and what are the key concerns?

Answer: Run as non-root: set runAsNonRoot: true and a specific runAsUser UID. Read-only root filesystem: set readOnlyRootFilesystem: true so the container cannot write to its own filesystem. Drop all Linux capabilities: capabilities: drop: [ALL] and only add what is strictly needed. Network policies: default-deny all traffic and explicitly allow only required paths. Secrets management: never use plain environment variables for secrets — mount from Key Vault via CSI driver. Image security: scan images in CI with Trivy or Snyk, use signed images, pull only from private registries. Pod Security Standards: enforce 'restricted' profile cluster-wide with admission controllers.

Q88. What is the CAP theorem and how does it apply to Azure services?

Answer: CAP: a distributed system can guarantee at most two of Consistency (every read returns the most recent write), Availability (every request gets a response), and Partition Tolerance (system continues during network failures). Since partitions always happen in distributed systems, the real choice is CP (consistent but may reject requests during partition) or AP (always responds but may return stale data). Azure SQL is CP — strongly consistent, but if the primary fails you wait for failover. Cosmos DB is configurable — choose from five consistency levels from strong (CP) to eventual (AP). Azure Service Bus is CP for message delivery.

Q89. What is Infrastructure as Code and which tools would you use on Azure?

Answer: Infrastructure as Code (IaC) manages cloud resources (VMs, databases, networks, queues) as code rather than through the portal. Benefits: version-controlled infrastructure, reproducible environments (dev and prod are identical), peer-reviewed changes, automated provisioning in CI/CD. Azure tools: Bicep (Azure-native, cleaner syntax than ARM templates, transpiles to ARM), Terraform

(multi-cloud, declarative, large ecosystem), Pulumi (multi-cloud, uses real programming languages including C#). For a .NET team on Azure, Bicep or Terraform are the most common choices.

SECTION 10: SYSTEM DESIGN

Q90. How would you design the project structure for a greenfield .NET 8 banking application?

Answer: Clean Architecture with four layers: Domain (entities, value objects, domain events, no external dependencies), Application (use cases, commands/queries via MediatR, interfaces), Infrastructure (EF Core, external APIs, messaging, concrete implementations), API (ASP.NET Core, middleware, configuration, dependency wiring). Separate test projects: unit tests, integration tests, and architecture tests (ArchUnitNET to enforce that Domain never references Infrastructure). Key day-one choices: nullable reference types enabled, EF Core for writes and Dapper for complex reads, FluentValidation, Serilog with structured logging, Azure Key Vault for secrets, xUnit and NSubstitute for testing.

Q91. When would you choose a modular monolith over microservices?

Answer: Start with a modular monolith when: the team is small (under 8 engineers), the domain boundaries are not yet understood (wrong microservice boundaries are expensive to fix), operational maturity is low (no observability or mature DevOps), or time to market is critical. A modular monolith has clean internal boundaries (separate modules for Trading, Settlement, Reference Data) but deploys as a single unit. Extract a service when a module proves it needs independent scaling, separate team ownership, or a different technology. This approach gives you time to learn domain boundaries before committing to service cuts.

Q92. How do you design caching for a high-traffic financial application?

Answer: Layered approach: browser or CDN for static assets and public read-only data with appropriate cache headers; application-level in-process cache (IMemoryCache) for single-instance, ultra-fast access; distributed cache (Redis) for shared state across multiple API instances; and database query cache as a last resort. Cache-aside pattern: check cache, on miss load from DB, populate cache. Invalidation: TTL for simple staleness tolerance; event-driven invalidation for data that changes and must be immediately consistent. Watch for cache stampede on popular misses — use locking or background refresh. Never cache sensitive financial position data without careful consideration of consistency requirements.

Q93. What is idempotency and how do you implement it in a payment API?

Answer: An idempotent operation produces the same result if performed multiple times. Critical for payments: network failures cause retries, and retries must not double-charge. Implementation: client sends a unique Idempotency-Key header per logical operation. Server checks if it has seen this key: if yes and completed, return the stored result; if yes and in-progress, return 409; if unseen, process, store the result keyed by the idempotency key with a TTL (24-48 hours), return the result. Redis is ideal for the store — atomic operations prevent race conditions. The key must be tied to the user to prevent cross-user replay attacks.

Q94. Explain the Saga pattern for distributed transactions.

Answer: A Saga is a sequence of local transactions where each step publishes an event or command to trigger the next, and failed steps trigger compensating transactions to undo previous steps. Two styles: choreography (services react to events — OrderPlaced triggers PaymentService which publishes PaymentSucceeded which triggers InventoryService) and orchestration (a central coordinator calls each service in sequence and handles failures by calling compensating actions). Choreography is decoupled but hard to trace. Orchestration is easier to reason about but adds a coordinator component. Key insight: compensating transactions are not always perfect reversals — you can refund a payment but cannot un-send an email.

Q95. How would you design a global translation engine that supports 100,000+ concurrent users?

Answer: Accept translation requests through load-balanced API gateways (Azure API Management). Enqueue each request to a distributed message queue (Azure Service Bus). Stateless worker services (AKS pods) pull from the queue and call Azure Cognitive Services for translation. Horizontal auto-scaling of workers based on queue depth. Cache frequent translations in Redis with TTL — many requests translate the same common phrases. Store results in Azure Blob Storage for durable retrieval. For truly real-time needs (live document translation), use streaming via gRPC or WebSockets. Key design elements: async decoupling between acceptance and processing, stateless workers for easy scaling, caching to reduce API calls and cost, and observability with distributed tracing for request tracking across the pipeline.

Q96. How do you design a multitenant, real-time risk management system?

Answer: Tenant isolation: every database table has a TenantId column. EF Core global query filters automatically append WHERE TenantId = @current to all queries. Extract TenantId from the JWT claims after authentication. Defense-in-depth: SQL Server row-level security as a second layer. Real-time data: webhook receivers update position data as market prices change; Azure Service Bus delivers position update events; consumers update in-memory risk snapshots in Redis. APIs serve the current risk snapshot from Redis rather than querying the database on every request. Multitenancy: each tenant's calculation parameters are configurable; risk limits and thresholds are stored per-tenant and cached.

Q97. How would you approach migrating a monolithic .NET Framework application to .NET 8 microservices?

Answer: Strangler Fig approach. Place Azure API Management in front of the monolith as a gateway. New features are built as .NET 8 microservices behind the gateway. Identify the first module to extract: pick the least coupled, most independently valuable piece. Build it as a separate .NET 8 service in a Docker container. Update the gateway to route relevant traffic to the new service. The monolith still serves everything else. Next: extract the data. Start with a shared database (temporary compromise). Gradually move the extracted service to its own database, using async events via Service Bus for data synchronization between services. Repeat for the next module. Each extraction reduces the monolith until it can be retired.

Q98. How do you handle database migrations in a continuous deployment pipeline?

Answer: Every schema change is a numbered migration script committed to source control alongside the application code. The CI pipeline runs migrations against a test database before merging. Migrations must be backward compatible: the new code must work with both the old schema and the new schema (for the duration of a rolling deployment). Deploy order: run migrations first, then deploy new application code. Never run migrations as part of application startup (EnsureCreated, Migrate in

Program.cs) — this breaks rolling deployments and adds startup latency. Use a separate pipeline stage or init container for migrations. Keep a rollback script for every migration and test it regularly.

Q99. How do you design observability for a microservices architecture?

Answer: Three pillars. Logs: structured logging with Serilog, include a correlation ID in every log entry so you can trace a request across services. Aggregate in Azure Application Insights or ELK. Metrics: request rate, error rate, latency percentiles (p50, p95, p99), queue depth, business metrics (trades per second, payment success rate). Expose via Prometheus or Application Insights custom metrics. Traces: distributed tracing with OpenTelemetry, propagate trace context (W3C TraceContext) across HTTP calls and message queues. Visualise in Application Insights or Jaeger. Alerting: set SLOs (error rate below 0.1%, p95 latency below 500ms) and alert when they breach. Health checks on every service: /health (liveness) and /health/ready (readiness for Kubernetes probes).

Q100. How would you implement a proof-of-concept using LangChain and Azure AI for document intelligence in a financial context?

Answer: Use Azure Document Intelligence (Form Recognizer) to extract structured data from documents (trade confirmations, KYC documents, contracts). Pass extracted text to an LLM via LangChain for semantic understanding: classify document type, extract key entities (counterparty, trade date, notional, currency), flag anomalies. Build a RAG (Retrieval Augmented Generation) pipeline: embed extracted text with Azure OpenAI embeddings, store in a vector database (Azure Cognitive Search with vector support), and allow natural language queries over the document corpus. Critical for banking: the LLM assists humans, it does not make autonomous decisions. All outputs are reviewed and auditable. No raw customer data is sent to external LLM endpoints — use Azure OpenAI on Azure with data residency guarantees. Log every LLM call with inputs and outputs for audit purposes.